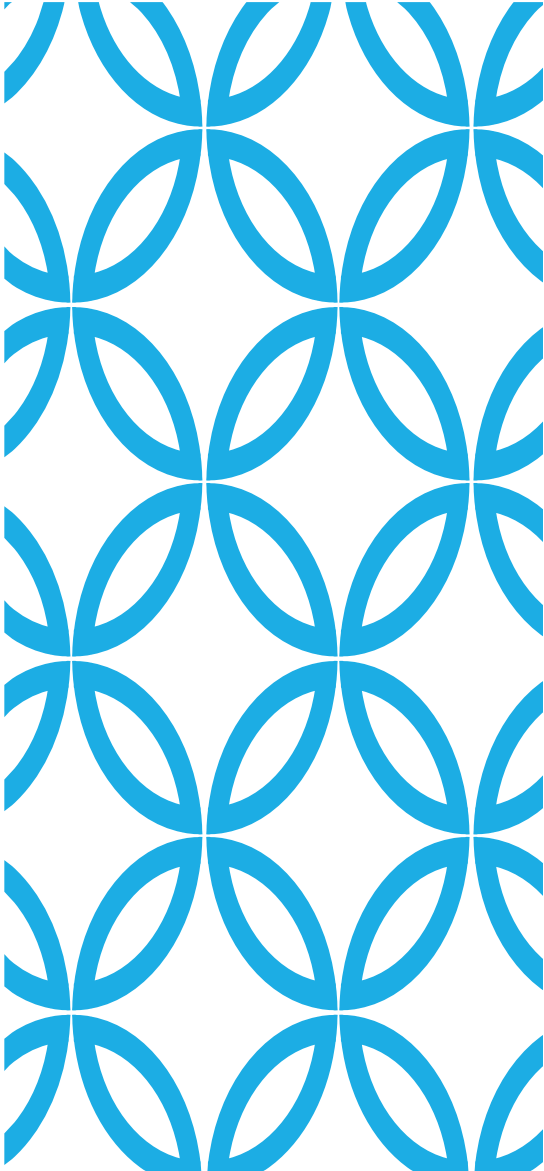




UNIDAD 2

Conceptos de Lenguaje de Bajo Nivel. Instrucciones

PROGRAMAS. LENGUAJES DE PROGRAMACIÓN

Los computadores ejecutan programas, los cuales son secuencia de operaciones conducentes a resolver un problema determinado.



Características de los programas

Están compuestos por secuencias de instrucciones o sentencias.

Se escriben utilizando una notación formal conveniente.

Pueden ser escritos por personas (programadores), o bien pueden ser generados automáticamente mediante una herramienta adecuada.

Un programa en ejecución se encuentra cargado en memoria principal.



Lenguaje de programación: una notación formal para describir algoritmos o funciones que serán ejecutadas por un computador

LENGUAJE DE PROGRAMA CIÓN DE BAJO NIVEL

Un lenguaje de programación de características **bajo nivel** es aquel en el que sus instrucciones

ejercen un control
directo sobre el
hardware y

están condicionados
por la estructura física
de las computadoras
que lo soportan



Se utiliza este tipo de lenguajes para programar tareas críticas de los sistemas operativos, de aplicaciones en tiempo real o controladores de dispositivos

LENGUAJE DE PROGRAMA CIÓN DE BAJO NIVEL

La estructura de los lenguajes son como sigue:

- ▣ **Lenguaje Máquina** - El lenguaje de máquina o código máquina es el sistema de códigos directamente interpretable por un circuito micro programable, como el microprocesador de una computadora. Este lenguaje está compuesto por un conjunto de instrucciones que determinan acciones a ser tomadas por la máquina. Un programa consiste en una cadena de estas instrucciones más un conjunto de datos sobre el cual se trabaja. Estas instrucciones son normalmente ejecutadas en secuencia, con eventuales cambios de flujo causados por el propio programa o eventos externos. El lenguaje de máquina es específico de la arquitectura de la máquina.
- ▣ **Lenguajes ensambladores** - También denominados nemotécnicos o nemónicos, no son ya programas ejecutables directamente por el ordenador, sino textos de código fuente que necesitan de alguna herramienta para su conversión a lenguaje máquina, son los programas llamados ensambladores. Sus instrucciones suelen ser una denominación abreviada de la instrucción máquina que simbolizan, y tienen una correspondencia casi directa a las instrucciones máquina que representan. El código resultante de la ejecución del programa ensamblador generaría un código binario ejecutable.

LENGUAJE DE PROGRAMA CIÓN DE BAJO NIVEL

En este tipo de lenguajes se trabaja a nivel de instrucciones, es decir, su programación es al más fino detalle, además, está completamente orientado a la máquina.

- Adaptación - Máxima entre programación y aprovechamiento del recurso de la máquina.
- Velocidad - Máxima al contar con un acceso directo a los recursos, sin capas intermedias.
- Portabilidad - Mínima por estar restringido a las especificaciones del fabricante.
- Abstracción - Mínima por depender completamente de la técnica del hardware.
- Uso - Requiere de la máxima atención y de una organización estructurada en base a los planos del hardware y del objetivo del software.

LENGUAJE DE PROGRAMA CIÓN DE ALTO NIVEL

La codificación de programas en binario es conveniente y natural para la circuitería del computador, pero es difícil para un programador humano.

El lenguaje ensamblador surgió para facilitar la escritura de programas de

computador.

- Es un lenguaje simbólico que da nombres a las instrucciones de máquina, y permite dar nombres a posiciones de memoria que contienen instrucciones o datos.

Los lenguajes de alto nivel facilitan la tarea de los programadores, ya que se encuentran más próximos a la forma de pensar de los humanos.

La programación en lenguajes de alto nivel es más productiva, ya que los programas son más cortos (en cuanto a líneas de código).

Hoy en día la práctica totalidad de los programadores trabaja utilizando lenguajes de alto nivel.

LENGUAJE DE PROGRAMA CIÓN

Lenguajes de bajo nivel: cercanos a la arquitectura de la máquina.

Lenguajes de alto nivel: cercanos a la forma de pensar del programador.

Lenguaje máquina: el único que la circuitería de la máquina es capaz de interpretar.

- Sus instrucciones se encuentran codificadas en binario.

Lenguajes simbólicos: no son directamente interpretables por la circuitería de la máquina.

- Se codifican mediante símbolos alfanuméricos, de puntuación, paréntesis, separadores, etc.

LENGUAJE ENSAMBLADOR

El lenguaje ensamblador (o lenguaje de ensamble, *assembly language*) es la representación simbólica de la codificación binaria de un computador.

Códigos de operación representados mediante códigos nemotécnicos.

Datos y referencias codificadas mediante nombres simbólicos (símbolos o etiquetas).

Existe una correspondencia biunívoca entre las instrucciones de máquina y las instrucciones de un lenguaje ensamblador.

Cada instrucción ensamblador es una codificación simbólica de una instrucción de máquina.

El lenguaje ensamblador debe ser traducido a lenguaje máquina para poder ser interpretado y ejecutado directamente por el computador.

LENGUAJE DE PROGRAMACIÓN

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

C compiler

Assembly
language
program
(for MIPS)

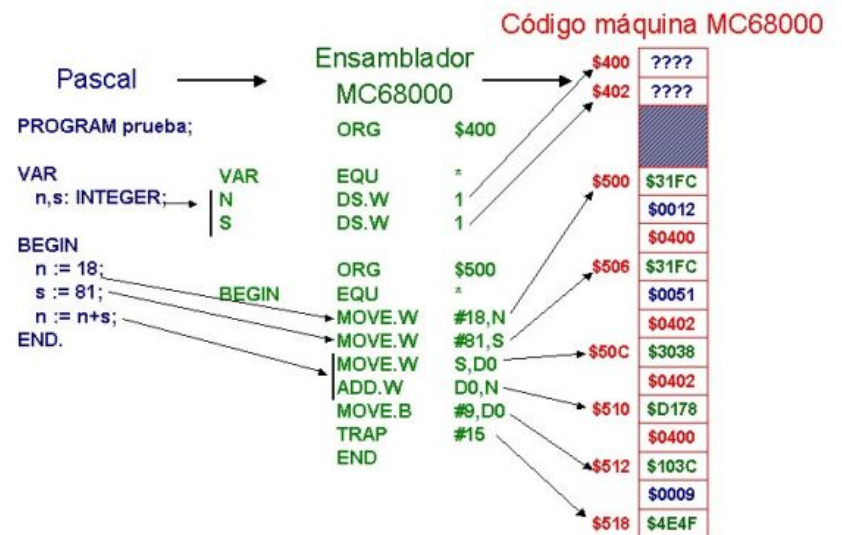
```
swap:
  muli $2, $5, 4
  add $2, $4, $2
  lw  $15, 0($2)
  lw  $16, 4($2)
  sw  $16, 0($2)
  sw  $15, 4($2)
  jr  $31
```

Assembler

Binary machine
language
program
(for MIPS)

```
000000001010000100000000000011000
00000000100011100001100000100001
10001100011000100000000000000000
100011001111100100000000000000100
10101100111110010000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```

LENGUAJE DE PROGRAMACIÓN



LENGUAJE DE PROGRAMACIÓN

La circuitería del computador tan solo comprende los programas escritos en lenguaje máquina.



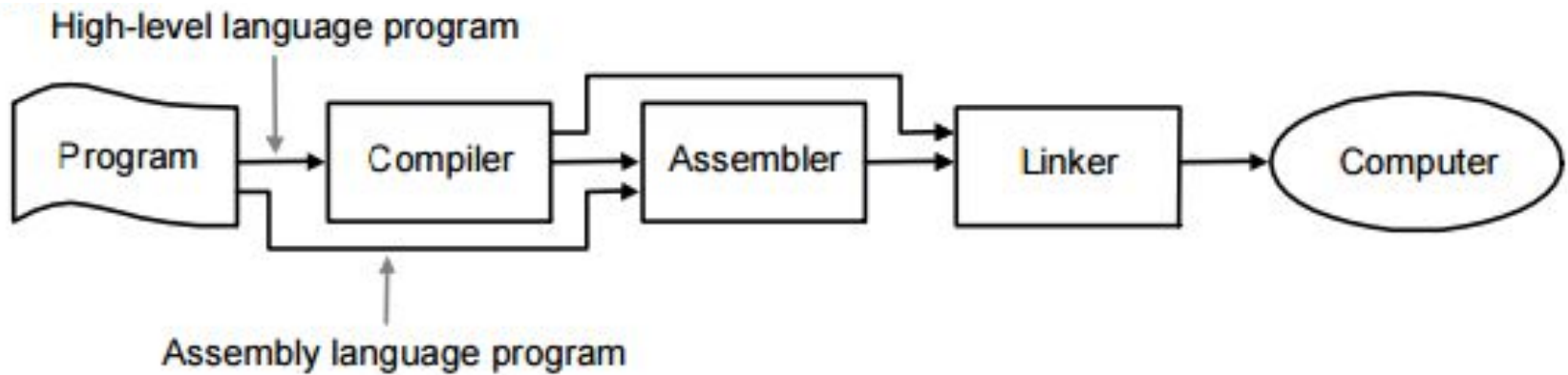
Los programas escritos en lenguajes simbólicos deben ser traducidos a lenguaje máquina antes de ser ejecutados.

Compilador (*Compiler*): traductor de lenguaje de alto nivel a lenguaje ensamblador. Hoy día los compiladores pueden traducir los programas directamente a lenguaje máquina.

Ensamblador (*Assembler*): traductor de lenguaje ensamblador a lenguaje máquina.

Montador (*Linker*): crea el código máquina ejecutable final.

Cargador (*Loader*): carga el código ejecutable en memoria y lo prepara para su ejecución.



LENGUAJE DE PROGRAMACIÓN

CARGA Y EJECUCIÓN

Consiste en la transferencia del programa ejecutable a la memoria del computador desde el fichero en disco, y en el posterior lanzamiento de su ejecución.

El programa ejecutable formado por instrucciones en lenguaje máquina se encuentra almacenado en posiciones consecutivas de memoria.

El Contador de Programa (PC) es un registro que contiene la dirección de la posición de memoria que contiene la instrucción que va a ser ejecutada a continuación.

Herramienta utilizada: cargador.

□ Pertenece al sistema operativo.

INSTRUCC IÓN

Un programa consta de una secuencia de instrucciones, cada una de las cuales especifica una acción particular. Una parte de la instrucción, llamada **código de operación**, señala la acción que va a ejecutarse.

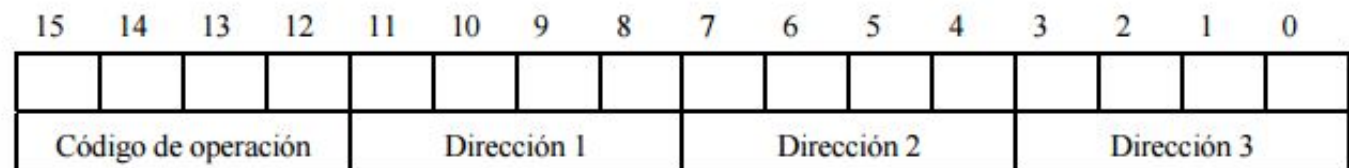
Muchas instrucciones contienen los datos que se usan, o bien especifican donde están. Todo lo relativo a la localización de los datos sobre los que se realiza la operación (es decir, la dirección de los operandos) se denomina **direccionamiento**.

El propósito de una instrucción es especificar la acción que debe ser realizada y los operandos sobre los que se efectúa esta acción. Dentro de esos operandos pueden quedar incluidos tanto los datos de la operación como los lugares donde deben depositarse los resultados que la instrucción pueda producir.

Sea una máquina cuyas instrucciones tienen 16 bits de longitud y las direcciones 4 bits.

Este formato es razonable para un tipo de máquina que tenga 16 registros.

Con este formato, tendremos en total 16 operaciones distintas (4 bits de código de operación) con 3 direcciones.



INSTRUCCIÓN

Ahora bien, si los diseñadores necesitan:

- 15 instrucciones de 3 direcciones
- 14 instrucciones de 2 direcciones
- 31 instrucciones de 1 dirección
- 16 instrucciones sin direcciones

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	-----											
0	0	0	1	-----											
---				-----											
1	1	0	1	-----											
1	1	1	0	-----											

INSTRUCCIÓN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	0	0	----			
1	1	1	1	1	1	1	0	0	0	0	1	----			
----				----				----				----			
1	1	1	1	1	1	1	0	1	1	1	0	----			
1	1	1	1	1	1	1	0	1	1	1	1	----			
1	1	1	1	1	1	1	1	0	0	0	0	----			
1	1	1	1	1	1	1	1	0	0	0	1	----			
----				----				----				----			
1	1	1	1	1	1	1	1	1	1	0	1	----			
1	1	1	1	1	1	1	1	1	1	1	0	----			

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	-----							
1	1	1	1	0	0	0	1	-----							
----				----				-----							
1	1	1	1	1	1	0	0	-----							
1	1	1	1	1	1	0	1	-----							

INSTRUCCIÓN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

INSTRUCCIÓN

INSTRUCCIONES

Repertorio o juego de instrucciones: conjunto de instrucciones de máquina que es capaz de ejecutar el computador. Debe ser:

Completo: debe permitir resolver cualquier problema.

Eficaz: los programas deben ser ejecutados en un tiempo razonable.



Está relacionado con:

El número de registros disponibles.

El tamaño de los datos.

Los modos de direccionamiento (maneras de acceder a los datos).



Clasificación de los computadores según su repertorio:

RISC: *Reduced Instruction Set Computer* (ej.: MIPS, PowerPC).

CISC: *Complex Instruction Set Computer* (80x86).

VLIW: *Very Long Instruction Word* (Itanium)

RISC, CISC Y VLIW

Attribute	RISC	CISC	VLIW
Stands For	Reduced Instruction Set Computing	Complex Instruction Set Computing	Very Large Instruction Word
Example	ARM processor	Intel, AMD processors	
Essence	Simple Instruction Set, hence longer and more complex programs	Complex Instruction Set with more instruction choices, makes programming easier	Multiple independent instructions clubbed into one instruction
Hardware	Simpler, since simple and fixed length instructions. No microcode is needed. Decoding is easy.	Complex since instructions are varying in length, can do multiple operations with single instruction. Needs microcode to break very complex instructions into simpler micro-operations. Decoding is difficult and complex.	Medium complex. VLIW instruction needs to be broken into several independent instructions. But each such instruction is simple (RISC instruction)
Pipeline	Simple and fast. Hence can be implemented with higher frequency	Complex and slow. Having high frequency is a challenge	Multiple simple pipelines executing in parallel
Software	Longer programs and complex compilers as logic needs to be implemented with limited number of instructions	Shorter and efficient programs as compiler has much more flexibility in choosing instructions for a task	Needs complex compilers who are able to find and put together multiple simpler instructions into a single instruction
Verification	Relatively easy since processor implementation is simpler	Relatively difficult as pipelines and implementation is complex	Moderate as complexity falls between that of RISC and CISC processor

INSTRUCCIONES

El repertorio de instrucciones debe especificar:

Formato de las instrucciones: tamaño fijo, variable o híbrido.

Localización de operandos y resultado, así como modos de direccionamiento de la memoria.

Tipos de datos y tamaños: Enteros (complemento a 2 o BCD, de 8, 16, 32 y 64 bits) y números en coma flotante de 32 y 64 bits (IEEE 754).

Operaciones soportadas: lógicas, aritméticas, etc.

Mecanismos de bifurcación: instrucciones de salto, subrutinas, etc., que modifican el flujo normal de ejecución.

Propiedades de las instrucciones:

Realizan una función única y sencilla.

Emplean un número fijo de operandos en una representación determinada.

Su codificación binaria es bastante sistemática, o al menos es recomendable que lo sea (con objeto de facilitar su decodificación).

Son auto-contenidas, es decir, contienen toda la información necesaria para su ejecución:

Código de operación.

Identificación de los operandos.

Destino.

Ubicación de la siguiente instrucción (normalmente implícita, pues suele ser la siguiente).

EJEMPLO — MIPS32

Máquina RISC (computador con repertorio de instrucciones reducido).

Ancho de palabra y tamaño de los buses: 32 bits.

Tamaño de los datos en las instrucciones:

- Bit (b): en muy pocas instrucciones.
- Byte (8 bits, B)
- Halfword (16 bits, H)
- Word (32 bits, W)
- Doubleword (64 bits, D)

Arquitectura de carga / almacenamiento:

- Antes de ser utilizado en una instrucción aritmética, todo dato debe ser cargado previamente en un registro de propósito general.
- Instrucciones aritméticas con 3 operandos de 32 bits en registros.

Esquema de bus único para memoria y E/S.

Modos de funcionamiento: usuario, núcleo (kernel), supervisor y depuración.

EJEMPLO — MIPS32

Tamaño de las instrucciones: $32 \text{ bits} = 4 \text{ bytes}$.

Número de operandos: 0, 1, 2 o 3.

- ▢ Instrucciones de carga o almacenamiento: 2 operandos, uno en registro y el otro en memoria (de 8, 16 o 32 bits).
- ▢ Instrucciones aritméticas o lógicas: 3 operandos de 32 bits en registros.
- ▢ Instrucciones aritméticas o lógicas con un inmediato: 3 operandos, dos de 32 bits en registros, y el tercero inmediato de 16 bits con extensión de signo (ceros o unos).

FORMATO DE INSTRUCCIÓN EN LENGUAJE MÁQUINA

Formato de instrucción: representación en binario de la misma.

- El formato de instrucción especifica el significado de cada uno de los bits que la constituyen.
- Longitud del formato de instrucción: número de bits que lo componen.

La información contenida en el formato de la instrucción es:

- Código de operación (COP).
- Dirección de los operandos (OP1 y OP2).
- Dirección del resultado (RES).
- Dirección de la siguiente instrucción (casi siempre implícita).
- Tipos de representación de los operandos (casi siempre implícitos en el código de operación).
- Modificador (MD): suele completar al CO, y sirve para especificar ciertas particularidades de la instrucción:
 - Tamaño y tipo de los operandos.
 - A veces se usa para distinguir entre operaciones similares.



FORMATO DE INSTRUCCIÓN EN LENGUAJE E MÁQUINA

Un computador contendrá instrucciones con diferentes formatos (= no todas las instrucciones del repertorio de un computador tienen el mismo formato).

- Un computador dispone de pocos formatos de instrucción diferentes para simplificar su decodificación.
- Los formatos son sistemáticos: campos del mismo tipo suelen ocupar la misma longitud y la misma posición.

El código de operación:

- Permite distinguir entre los distintos formatos de instrucción de un computador.
- Indica la longitud y formato de los operandos (a veces en campos modificadores asociados al mismo).

Las longitudes de los formatos son fracción o múltiplo del tamaño de la palabra del computador.

- Para acortar la longitud de los formatos se utiliza **direccionamiento implícito**: ninguna instrucción, salvo las de salto o bifurcación, contiene la dirección de la siguiente instrucción que se va a ejecutar

FORMATO DE INSTRUCCIÓN EN LENGUAJE E MÁQUINA

MIPS presenta tres formatos básicos de instrucción (32 bits):

- Tipo R, o instrucciones de registro.
- Tipo I, instrucciones de transferencia de datos o ramificación condicional.
- Tipo J, o de salto incondicional.

TIPOS DE INSTRUCCIONES

Transferencia de datos: mover, cargar, almacenar.

Aritméticas para enteros: sumar, restar, multiplicar, dividir, etc.

Aritméticas para coma flotante: sumar, restar, multiplicar, dividir, etc.

Operaciones lógicas: and, or, not, xor, etc.

Activación condicional: si es r1 igual r2 entonces, saltar; si r1 es menor que r2, entonces saltar, etc.

Instrucciones de rotación y desplazamiento: desplazar hacia la

izquierda/derecha, etc.

Instrucciones de control de programa: saltar, saltar si menor, saltar y enlazar dirección, etc.

Instrucciones de control de sistema: provocar excepción, llamar al sistema operativo, etc.

TIPOS DE INSTRUCCIONES

Instrucciones aritméticas y de desplazamiento para enteros e instrucciones lógicas

Sintaxis	T	Descripción
add rd,rs,rt	I	rd = rs+rt
addi rd,rs,inm16	I	rd = rs+ext_signo(inm16,32)
addu rd,rs,rt	I	rd = rs+rt
addiu rd,rs,inm16	I	rd = rs+ext_signo(inm16,32)
div rs,rt	I	lo = rs/rt; hi = rem(rs/rt)
divu rs,rt	I	lo = rs/rt; hi = rem(rs/rt)
mult rs,rt	I	hi-lo = rs1*s2
multu rs,rt	I	hi-lo = rs1*s2
sub rd,rs,rt	I	rd = rs-rt
subu rd,rs,rt	I	rd = rs-rt
and rd,rs,rt	I	rd = rs AND rt
andi rd,rs,inm16	I	rd = rs AND ext_ceros(inm16,32)
nor rd,rs,rt	I	rd = rs NOR rt
or rd,rs,rt	I	rd = rs OR rt
ori rd,rs,inm16	I	rd = rs OR ext_ceros(inm16,32)
xor rd,rs,rt	I	rd = rs XOR rt
xori rd,rs,inm16	I	rd = rs XOR ext_ceros(inm16,32)
sll rd,rt,shamt5	I	rd = desp_log(rt,shamt5,izquierda)
sllv rd,rt,rs	I	rd = desp_log(rt,rs4..0,izquierda)
sra rd,rt,shamt5	I	rd = desp_arit(rt,shamt5,derecha)
srav rd,rt,rs	I	rd = desp_arit(rt,rs4..0,derecha)
srl rd,rt,shamt5	I	rd = desp_log(rt,shamt5,derecha)
srlv rd,rt,rs	I	rd = desp_log(rt,rs4..0,derecha)

TIPOS DE INSTRUCCIONES

Sintaxis	T	Descripción
lb rt, desp(rs)	I	rt = ext_signo(Mem[desp+rs] _{7..0} , 32)
lbu rt, desp(rs)	I	rt = ext_ceros(Mem[desp+rs] _{7..0} , 32)
lh rt, desp(rs)	I	rt = ext_signo(Mem[desp+rs] _{15..0} , 32)
lhu rt, desp(rs)	I	rt = ext_ceros(Mem[desp+rs] _{15..0} , 32)
lui rd, inml6	I	rd _{31..16} = inml6; rd _{15..0} = 0
lw rt, desp(rs)	I	rt = Mem[desp+rs]
lwcZ rt, desp(rs)	I	coprocesadorZ(rt) = Mem[desp+rs]
lwl rt, desp(rs)	I	rt _{31..16} = Mem[desp+rs]
lwr rt, desp(rs)	I	rt _{15..0} = Mem[desp+rs]
sb rt, desp(rs)	I	Mem[desp+rs] = rt _{7..0}
sh rt, desp(rs)	I	Mem[desp+rs] = rt _{15..0}
sw rt, desp(rs)	I	Mem[desp+rs] = rt
swcZ rt, desp(rs)	I	Mem[desp+rs] = coprocesadorZ(rt)
swl rt, desp(rs)	I	Mem[desp+rs] = rt _{31..16}
swr rt, desp(rs)	I	Mem[desp+rs] = rt _{15..0}
mfcZ rt, rd	I	rd = rt; rd: registro UCP; rt: registro del coprocesador Z
mfhi rd	I	rd = hi
mflo rd	I	rd = lo
mtcZ rd, rt	I	rd = rt; rd: registro UCP; rt: registro del coprocesador Z
mthi rd	I	hi = rd
mtlo rd	I	lo = rd

TIPOS DE INSTRUCCIONES

Instrucciones de activación condicional, control de programa y misceláneas

Sintaxis	T	Descripción
slt rd,rs,rt	I	Si $rs < rt$, $rd = 1$; si no, $rd = 0$
slti rd,rs,inm16	I	Si $rs < \text{ext_signo}(\text{inm16},32)$, $rd = 1$; si no, $rd = 0$
sltu rd,rs,rt	I	Si $rs < rt$, $rd = 1$; si no, $rd = 0$
sltiu rd,rs,inm16	I	Si $rs < \text{ext_signo}(\text{inm16},32)$, $rd = 1$; si no, $rd = 0$
bc2f etiqueta	I	Si $\text{flag}(\text{coprocesadorZ}) = 0$, ramificar a etiqueta
bc2t etiqueta	I	Si $\text{flag}(\text{coprocesadorZ}) = 1$, ramificar a etiqueta
beq rs,rt,etiqueta	I	Si $rs = rt$, ramificar a etiqueta
bgez rs,etiqueta	I	Si $rs \geq 0$, ramificar a etiqueta
bgezal rs,etiqueta	I	Si $rs \geq 0$, ramificar a etiqueta y enlazar ($\$ra = PC$)
bgtz rs,etiqueta	I	Si $rs > 0$, ramificar a etiqueta
blez rs,etiqueta	I	Si $rs \leq 0$, ramificar a etiqueta
bltz rs,etiqueta	I	Si $rs < 0$, ramificar a etiqueta
bltzal rs,etiqueta	I	Si $rs < 0$, ramificar a etiqueta y enlazar ($\$ra = PC$)
bne rs,rt,etiqueta	I	Si $rs \neq rt$, ramificar a etiqueta
j objetivo	I	$PC = PC31..28 \parallel (\text{objetivo} \ll 2)$
jal objetivo	I	$ra = PC$; $PC = PC31..28 \parallel (\text{objetivo} \ll 2)$
jalr rs,rd	I	$rd = PC$; $PC = rs$
jr rs	I	$PC = rs$
rfe	I	Restaurar desde excepción (Restaura el registro Status)
syscall	I	Llamada a un servicio del sistema ($\$v0$: número del servicio)
break codigo20	I	Provoca una excepción (código 1 reservado para el depurador)
nop	I	No operación

MODOS DE DIRECCIONAMIE NTO

Los operandos pueden ubicarse en los siguientes lugares:

- Dentro de la propia instrucción (operandos “inmediatos”):
 - En el registro de instrucción.
 - En palabras de extensión o ampliación.
- En registros visibles para el programador.
- En variables ubicadas en posiciones de memoria.

Los operandos no están contenidos en la instrucción de forma directa por:

- Ahorro de espacio.
- Empleo de código reubicable y reentrante.
- Hay datos formando estructuras más o menos complejas.

Los operandos pueden ser referenciados desde la instrucción de múltiples formas, dando lugar a los modos de direccionamiento

MODOS DE DIRECCIONAMIE NTO

Modo de direccionamiento: mecanismo que permite conocer la ubicación de un objeto (dato o instrucción).

Un computador debe disponer de varios modos de direccionamiento.

No todos los modos de direccionamiento están implementados en todos los computadores.

Los modos de direccionamiento disponibles están determinados por la arquitectura interna de la máquina y por el repertorio de instrucciones.

MODOS DE DIRECCIONAMIE NTO

Modos de direccionamiento:

- Direccionamiento directo a registro.
- Direccionamiento indirecto a registro con desplazamiento.
- Direccionamiento relativo a PC con desplazamiento.
- Direccionamiento inmediato.
- Direccionamiento pseudodirecto.
- Direccionamiento implícito.

MODOS DE DIRECCIONAMIENTO

Direccionamiento directo a registro

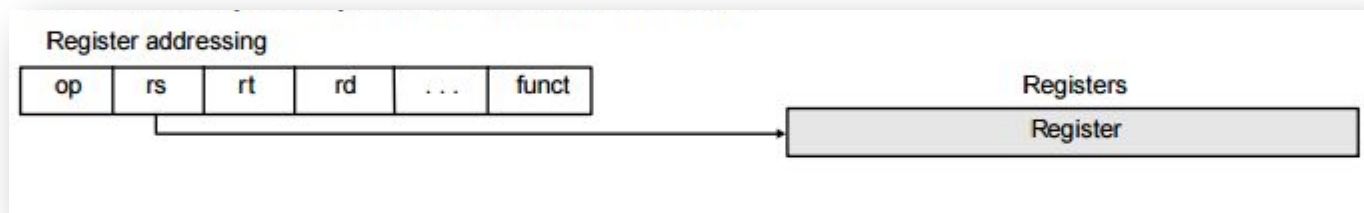
El campo tiene 5 bits, para operando fuente o destino

Notación: \$n

Los registros tienen alias, indicando la función para la que están dedicados según los convenios adoptados por programadores y compiladores.

Ejemplo: add \$16, \$17, \$18

□ suma el contenido del registro 17 con el registro 18 y lo almacena en el registro 16.



MODOS DE DIRECCIONAMIENTO

Direccionamiento indirecto a registro con desplazamiento

Campos:

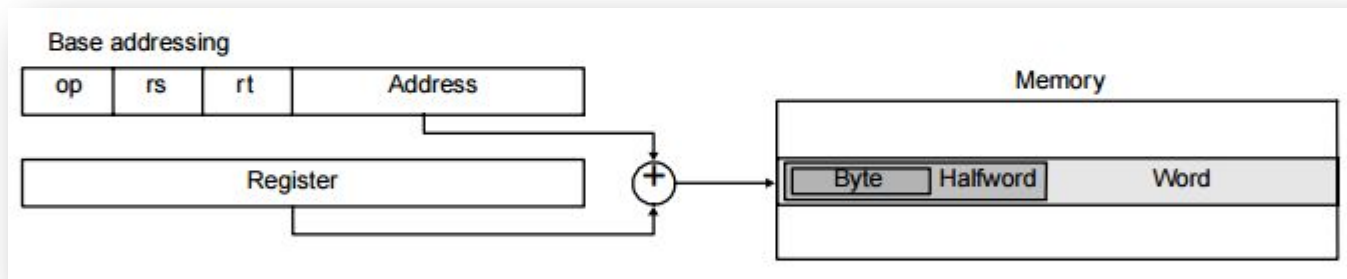
- Registro: 5 bits.
- Desplazamiento: 16 bits.

Permitido para operando fuente en almacenamientos y destino en cargas.

Notación: desplazamiento(\$n)

Ejemplo: lw \$16, 20(\$17)

- carga (“load”) en el registro 16 la palabra contenida 20 bytes más allá de la dirección almacenada en el registro 17.



MODOS DE DIRECCIONAMIENTO

Direccionamiento relativo a PC con desplazamiento

Campos:

▢ Desplazamiento: 16 bits.

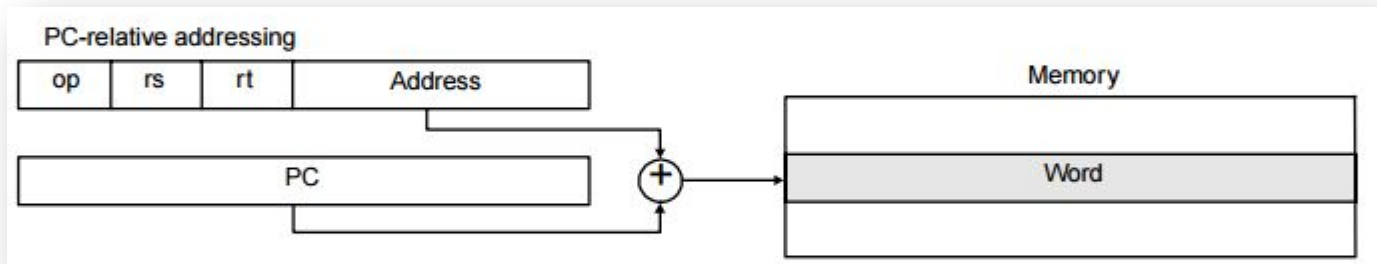
El desplazamiento se alinea a múltiplo de 4 bits y se extiende en signo a 32 bits para calcular la dirección efectiva.

Se utiliza para direcciones en bifurcaciones.

Notación: oculto en etiqueta.

Ejemplo: bne \$16, \$17, Fin

▢ Si el contenido del registro 16 es $\neq$ del 17, entonces salta a la dirección indicada por la etiqueta “Fin”



MODOS DE DIRECCIONAMIENTO

Direccionamiento inmediato

Campos:

- Inmediato de 16 bits.

Los datos inmediatos se extienden a 32 bits.

- En ciertas instrucciones se hace extensión de signo.
- En otras instrucciones se hace extensión con ceros.

También se usa para indicar la longitud de un desplazamiento.

- En este caso el inmediato es de 5 bits.

Permitido sólo para operandos fuente

Notación: dato (sin prefijo).

Ejemplo: `addi $16, $17, 1`

- suma una unidad al contenido del registro 17 y el resultado lo almacena en el registro 16.

Immediate addressing

op	rs	rt	Immediate
----	----	----	-----------

MODOS DE DIRECCIONAMIENTO

Direccionamiento pseudodirecto

Campos:

▢ Dirección de 26 bits.

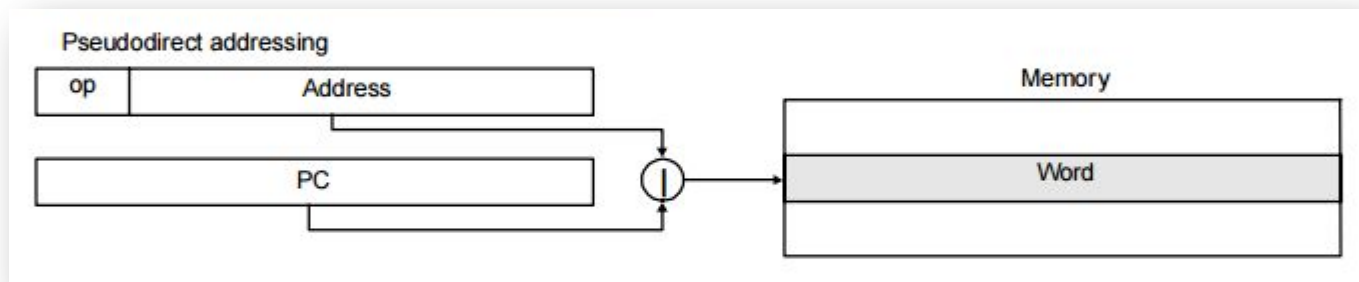
La dirección se desplaza dos bits a la izquierda, y se concatena a los cuatro primeros bits del contador de programa.

Se utiliza en instrucciones de salto.

Notación: dirección (sin prefijo)

Ejemplo: `j 2500`

▢ salta a la dirección de memoria correspondiente a la palabra 2500 (dirección 10000).



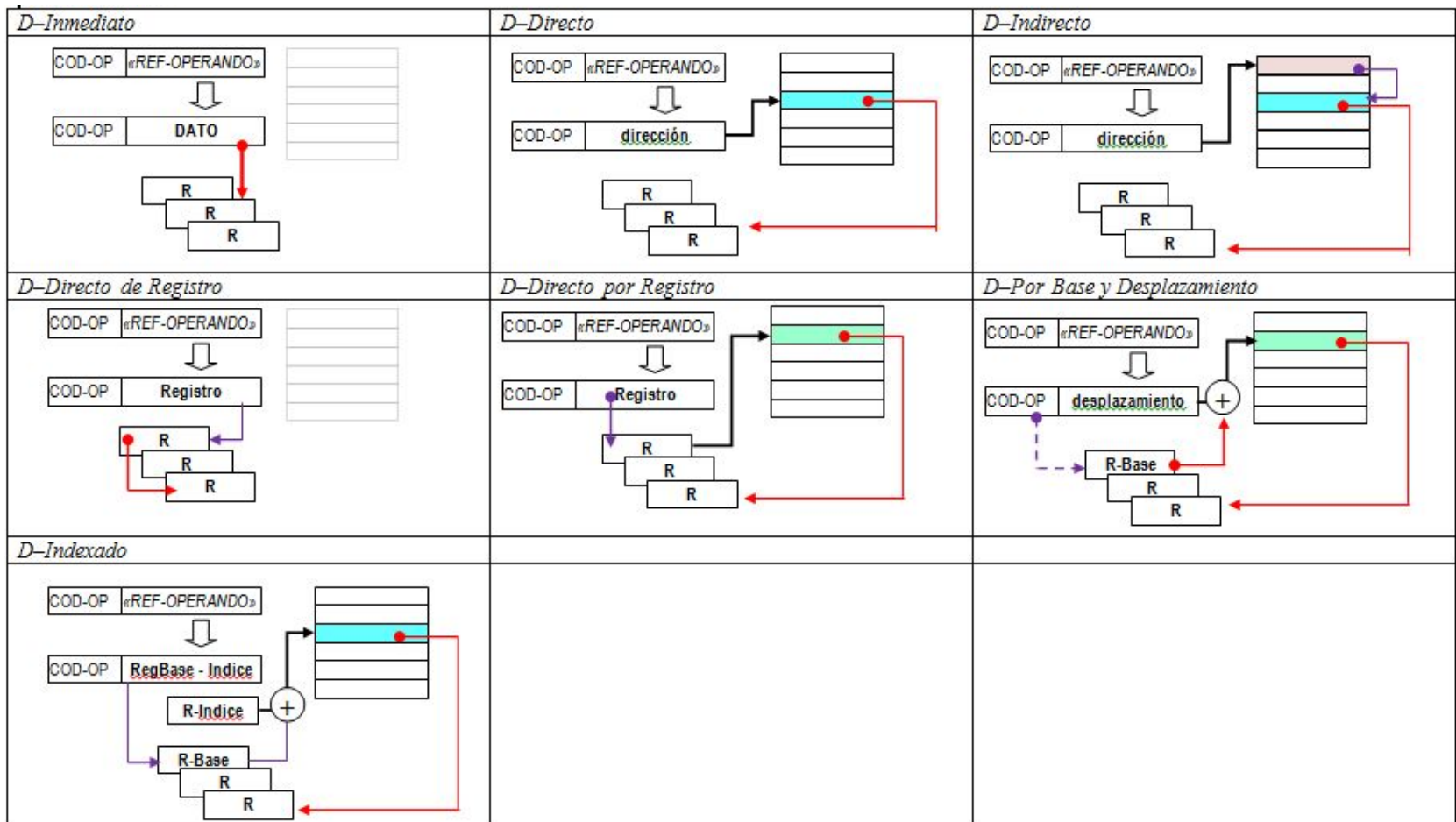
MODOS DE DIRECCIONAMIENTO

Direccionamiento implícito

Un operando tiene direccionamiento implícito cuando está determinado por el código de operación.

Ejemplo: jal 2500

- almacena la dirección de la siguiente instrucción del programa ($PC + 4$) en el registro de retorno \$ra y salta a la dirección de memoria correspondiente a la palabra 2500 (dirección 10000), donde empieza una subrutina



MODOS DE DIRECCIONAMIENTO

EXTRA

Si sólo podemos sumar dos registros en cada instrucción, ¿cómo hacemos operaciones más complicadas?

➡ C:

```
int a, b, c, d, e;  
a = ( b + c ) - ( d + e );
```

➡ MIPS:

```
# Suponiendo que los datos a, b, c, d, e  
# están asignados a los registros  
# $s0, $s1, $s2, $s3 y $s4, respectivamente:  
add $t0, $s1, $s2 # $t0 = $s1 + $s2  
add $t1, $s3, $s4 # $t1 = $s3 + $s4  
sub $s0, $t0, $t1 # $s0 = $t0 - $t1
```

EXTRA

¿cómo hacemos operaciones más complicadas?

➡ C:

```
if (i==j) goto L1;  
f = g + h;  
L1: f = f - i;
```

➡ MIPS:

Suponiendo que a las 5 variables de f a j les

corresponden los registros de \$s0 a \$s4

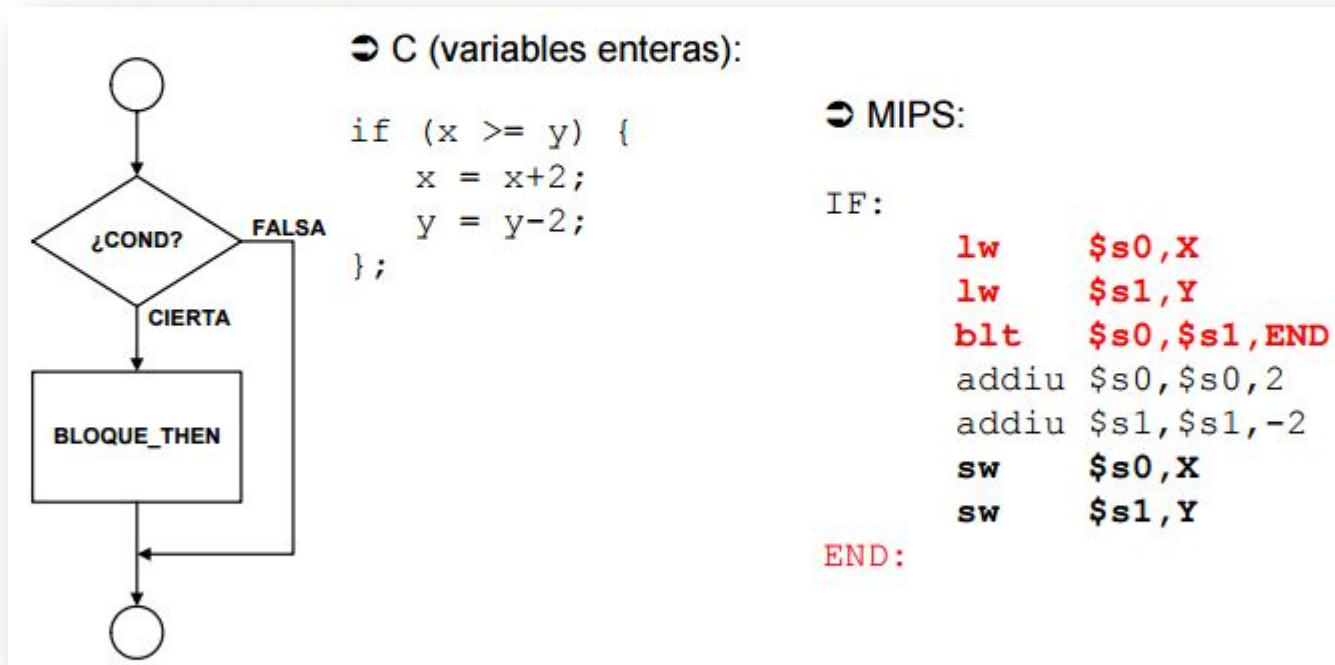
```
beq $s3, $s4, L1 # if ($s3 == $s4) goto L1
```

```
add $s0, $s1, $s2 # f = g + h
```

```
L1: sub $s0, $s0, $s3 # L1: f = f - i
```

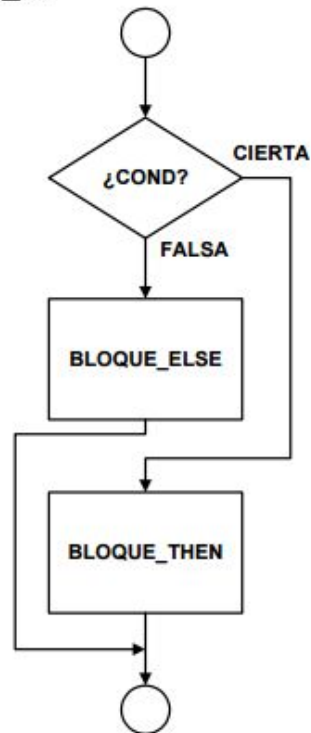
EXTRA

¿cómo hacemos operaciones más complicadas?



EXTRA

¿cómo hacemos operaciones más complicadas?



⇒ C (variables enteras):

```
if (x >= y) {  
    x = x+2; y = y+2;  
}  
else {  
    x = x-2; y = y-2;  
}
```

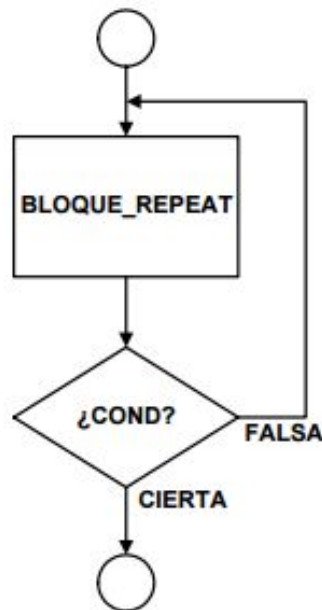
⇒ MIPS:

IF:

```
lw    $s0,X  
lw    $s1,Y  
bge   $s0,$s1,THEN  
ELSE: addiu $s0,$s0,-2  
      addiu $s1,$s1,2  
      sw    $s0,X  
      sw    $s1,Y  
      j     END  
THEN: addiu $s0,$s0,2  
      addiu $s1,$s1,-2  
      sw    $s0,X  
      sw    $s1,Y  
END:
```

EXTRA

¿cómo hacemos operaciones más complicadas?



➔ C (variables enteras):

```
a = 81;  
b = 18;  
do {  
    mcd = b;  
    resto = a % b;  
    a = b;  
    b = resto;  
} while (resto <> 0);
```

➔ MIPS:

```
# Variable A: $s0  
# Variable B: $s1  
# Variable MCD: $s2  
# Variable RESTO: $s3
```

```
li    $s0,81
```

```
li    $s1,18
```

REPEAT:

```
move  $s2,$s1
```

```
div   $s0,$s1
```

```
mfhi  $s3
```

```
move  $s0,$s1
```

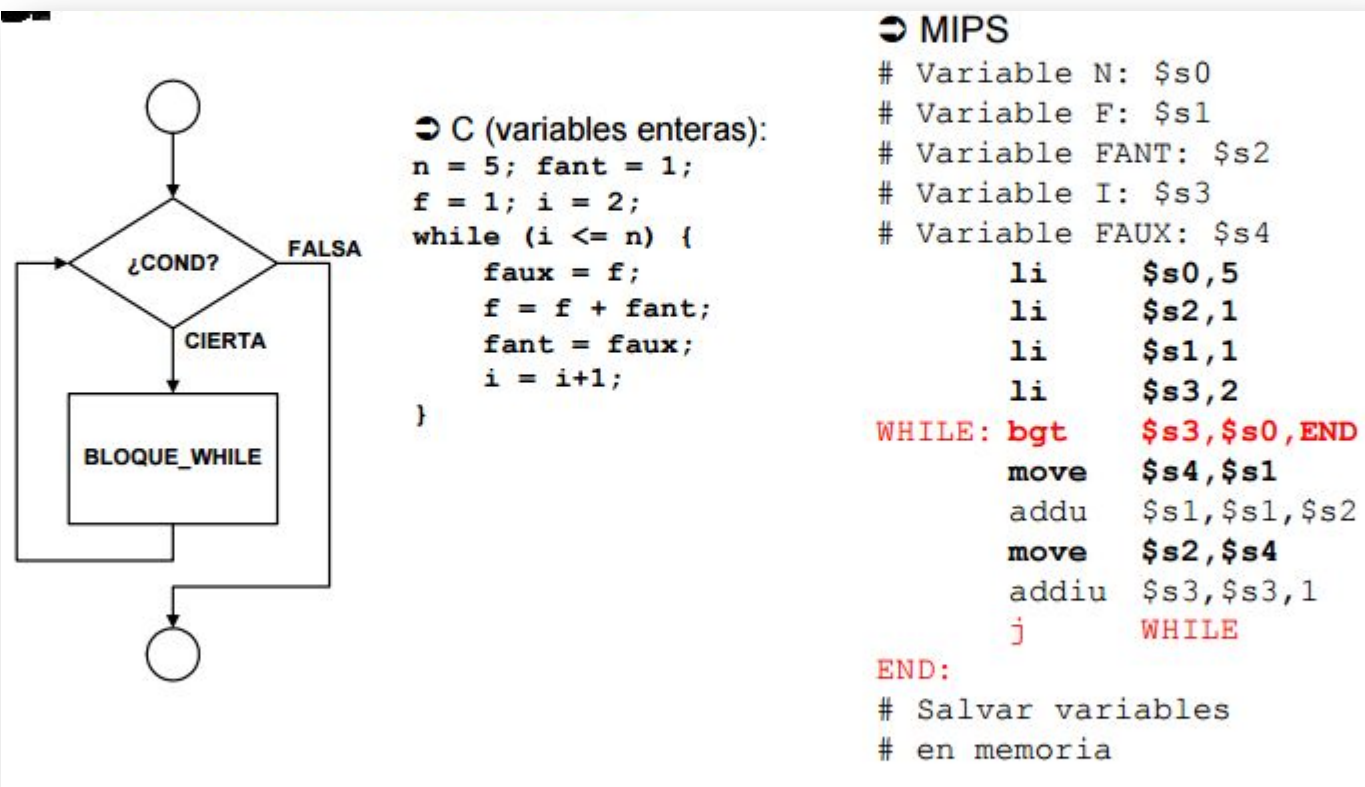
```
move  $s1,$s3
```

UNTIL: **bnez \$s3,REPEAT**

```
# Habría que salvar  
# las variables A, B, MCD  
# y RESTO en memoria
```

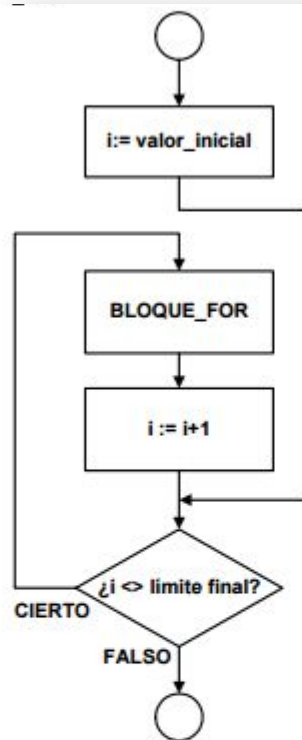
EXTRA

¿cómo hacemos operaciones más complicadas?



EXTRA

¿cómo hacemos operaciones más complicadas?



➤ C (variables enteras):

```
n = 5;
fant = 1;
f = 1;
for (i=2; i<=n; i++) {
    faux := f;
    f := f + fant;
    fant := faux;
}
```

➤ MIPS (versión PASCAL):

```
# Variable N: $s0
# Variable F: $s1
# Variable FANT: $s2
# Variable I: $s3
# Variable FAUX: $s4

        li    $s0,5
        li    $s2,1
        li    $s1,1
FOR:     move  $t0,$s0
        li    $s3,2
        bgt   $s3,$t0,END
        j     BODY
INC:     addiu  $s3,$s3,1
BODY:    move  $s4,$s1
        addu   $s1,$s1,$s2
        move   $s2,$s4
COND:    bne   $s3,$t0,INC
END:
# Salvar variables en memoria
```